Cache Simulator in C++

Project Overview

This project is a **Cache Memory Simulator** developed in **C++** for the subject of **Computer Architecture**. The simulator demonstrates how cache memory works in modern computer systems and helps students understand memory hierarchy, cache hits, cache misses, and different mapping techniques.

The simulator supports:

- Direct Mapping
- Fully Associative Mapping
- Set Associative Mapping
- FIFO Replacement Policy
- LRU Replacement Policy
- Cache Hit and Miss Calculation
- Memory Address Simulation

This project is suitable for:

- Computer Architecture course assignments
- Operating Systems and Memory Management studies
- GitHub portfolio projects
- Educational demonstrations

Features

Direct Mapping

Each memory block maps to exactly one cache line.

Fully Associative Mapping

Any memory block can be stored in any cache line.

Set Associative Mapping

Cache is divided into sets where each block can be stored within a specific set.

Replacement Policies

- FIFO (First In First Out)

- LRU (Least Recently Used)

Performance Statistics

The simulator calculates:

- Total memory accesses
- Cache hits
- Cache misses
- Hit ratio
- Miss ratio

Technologies Used

Technology	Purpose
C++	Core Programming Language
STL	Data Structures
VS Code / Dev C++	Development Environment
GitHub	Version Control

Project Structure

```
Cache-Simulator/
│
├── main.cpp
├── cache.h
├── cache.cpp
├── README.md
└── screenshots/
    ├── output1.png
    ├── output2.png
    └── output3.png
```

How Cache Works

Cache memory is a small high-speed memory located between CPU and RAM. It stores frequently used data to improve processing speed.

Memory Hierarchy

```
CPU Registers
↓
Cache Memory
↓
Main Memory (RAM)
↓
Secondary Storage
```

The closer memory is to the CPU, the faster it operates.

Algorithm

Direct Mapping Algorithm

1. Divide memory address into:

2. Tag

3. Index
4. Offset
5. Use index to locate cache line.
6. Compare tag values.
7. If tag matches → Cache Hit.
8. Otherwise → Cache Miss.

Sample C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int cache[4] = {-1, -1, -1, -1};
```

```
    int memory[] = {1, 2, 3, 1, 4, 2, 5};
    int hits = 0, misses = 0;

    for (int i = 0; i < 7; i++) {
        int index = memory[i] % 4;

        if (cache[index] == memory[i]) {
            hits++;
            cout << "Hit: " << memory[i] << endl;
        } else {
            misses++;
            cache[index] = memory[i];
            cout << "Miss: " << memory[i] << endl;
        }
    }

    cout << "\nTotal Hits: " << hits << endl;
    cout << "Total Misses: " << misses << endl;

    return 0;
}
```

Sample Output

```
Miss: 1
Miss: 2
Miss: 3
Hit: 1
Miss: 4
Hit: 2
Miss: 5

Total Hits: 2
Total Misses: 5
```

Installation Guide

Clone Repository

```
git clone https://github.com/yourusername/cache-simulator.git
```

Open Project

Open the project in:

- VS Code
- Dev C++
- CodeBlocks
- Visual Studio

Compile Program

```
g++ main.cpp -o cache
```

Run Program

```
./cache
```

Screenshots

Add screenshots of:

- Program execution
- Terminal output
- Cache simulation results
- Hit/Miss statistics

Example:

```
screenshots/output1.png
```

Applications of Cache Memory

- Faster data access
- CPU performance improvement
- Reduced memory access time
- Better multitasking performance
- Efficient execution of programs

Future Improvements

The project can be improved by adding:

- GUI Interface
- Graphical Visualization
- Multi-Level Cache (L1, L2, L3)
- Real-Time Simulation
- File Input Support
- Performance Graphs

Learning Outcomes

After completing this project, students will understand:

- Memory hierarchy
- Cache organization
- Mapping techniques
- Replacement algorithms
- Cache performance analysis

Conclusion

This Cache Simulator project provides a practical understanding of cache memory concepts used in computer architecture. It demonstrates how different mapping and replacement techniques affect system performance.

The project is simple, educational, and suitable for academic submissions as well as GitHub portfolios.

Author

Saif Ali Khan
Computer Science Student

License